Prøveeksamen INF-0103

Les oppgaveteksten grundig og disponer tiden slik at du får tid til å svare på alle oppgavene.

I noen oppgaver kan det være nødvendig å tolke oppgaveteksten ved å gjøre noen antagelser. Gjør i så fall rede for hvilke antagelser du har gjort, men pass på å ikke gjøre antagelser som trivialiserer oppgaven.

Der du skal utvikle eller beskrive en algoritme anbefales det at du først beskriver algoritmen på et høyt abstraksjonsnivå, f.eks. med ord og figurer, før du går videre med detaljer og eventuell pseudokode.

Der du blir spurt om å implementere noe kan du skrive C-lignende pseudokode. Skrivefeil eller syntaxfeil trekker ikke ned.

Husk at du kan referere tilbake til funksjoner du tidligere har definert.

Oppgave 1 (100%)

I oppgavene nedenfor er du nødt til ta noen avgjørelser og gjøre noen antakelser. Beskriv disse.

a) Implementer en datatype `struct rect` som holder et rektangel (altså en firesidet figur med kun rette vinkler).

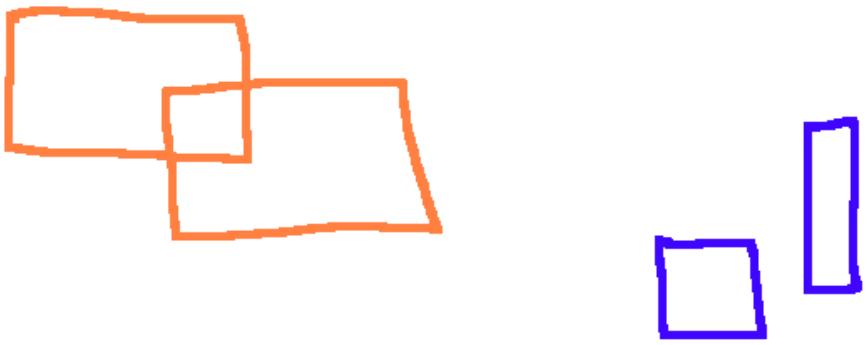


Figure 1: De orange rektanglene overlapper, de blå gjør ikke.

b) Anta at vi har gjort `typedef struct rect rect_t;` slik at vi slipper å skrive `struct rect` hele tiden. Skriv en funksjon som tar inn to `rect_t` og sjekker om de overlapper.

c) Skriv en funksjon som tar inn et array av `rect_t` og sjekker at ingen av dem overlapper.

Sensorveiledning

Siden dette er "ingen hjelpemidler" skal ikke studenten straffes for skrivefeil og syntaksfeil. Det trekker allikevel litt ned hvis koden er rotete, variabelnavn ikke gir mening, osv. Deloppgave **b)** er nok vanskeligst.

a)

Ikke så mange avgjørelser å ta her. Man kan spesifisere rektangelet på litt forskjellige måter (fire hjørner? ett hjørne + bredde/høyde?). Man må også velge om det skal være flyttallskoordinater eller heltall uten at det ene er mer rett enn det andre.

De som velger å representere rektangelet som hjørne + bredde + høyde bør gjøre rede for hvilket hjørne det er snakk om.

```
// jeg representerer rektangelet som nedre venstre hjørne + bredde + høyde og
// antar heltallskoordinater
struct rect {
    // hjørnekoordinater
    int x;
    int y;

    // width, heigth
    int w;
    int h;
}
```

b)

Det er ikke viktig å dekke alle corner cases. Hovedpoenget er at hvis det ene rektangelet inneholder minst ett hjørne av det andre har vi overlapp. Det er antakelig lett å gjøre litt feil i bokholderiet med if-testene: trenger ikke være veldig streng med det hvis den rette ideen er der.

Funksjonen bør returnere noe fornuftig siden den skal/bør brukes i c).

```
// jeg vil returnere 0 hvis det ikke er overlapp og 1 hvis der er det.
// for å sjekke om to rektangler overlapper er det nok å sjekke om ett hjørne av
// det ene er inneholdt i det andre.
int check_overlap(rect_t a, rect_t b) {
    if (rect_in_rect(a, b) || rect_in_rect(b, a)) return 1;
    return 0;
}

// siden det er mulig for det ene rect_t å være 100% inneholdt i det andre må
// jeg sjekke a mot b og b mot a. jeg velger derfor å bruke denne hjelpefunksjonen
// for hvert hjørne på a må vi sjekke at det ikke er inni b. f.eks for (a.x, a.y):
// (b.x  < a.x  < b.x + w) OG (b.y < a.y < b.y + h)
int rect_in_rect(rect_t a, rect_t b) {
```

```
    // sjekker hvert hjørne av a mot b
    if (point_in_rect(a.x, a.y, b)) return 1;
    if (point_in_rect(a.x + a.w, a.y, b)) return 1;
    if (point_in_rect(a.x + a.w, a.y + h, b)) return 1;
    if (point_in_rect(a.x, a.y + h, b)) return 1;

    // kommer vi hit er det fordi ingen hjørner av a var inni b
    return 0;
}

// denne gjør det litt enklere
int point_in_rect(int x, int y, rect_t r) {
    if ((r.x < x && x < r.x + r.w) && (r.y < y && y < r.y + r.h)) return 1;
    return 0;
}
```

c) Her er det lov å anta at c) er løst selv om man ikke helt klarte den. Det er ikke nøye om de går for den marginalt enklere n^2 "sjekk alle mot alle" løsningen eller n(n+1)/2 løsningen der man kun sjekker akkurat de man trenger.

Teknisk sett må for n^2 løsningen passe på når `i == j`. Ikke en stor feil å glemme dette. Det er andre mulige slurvefeil med akkurat hvordan man gjør iterasjonen i det andre tilfellet som heller ikke bør trekke så veldig mye ned.

Her må antas noe om hvordan arrayet sendes inn i funksjonen. I utgangspunktet vet vi ikke hvor stort det er. Det er lov å bare "anta at vi vet" men da må de gjøre rede for det. Hvis dette ignoreres er det allikevel ikke en stor feil; hovedpoenget er at de får til iterasjonen.

```
// jeg bruker funksjonen fra b) for å sjekke alle par av rektangler i arrayet.
// den som kaller funksjonen må passere inn arraystørrelsen slik at vi vet hvor
// langt vi må iterere
int overlaps_in_array(rect_t *arr, int n) {
    // det er nok å finne et eneste overlapp slik at vi kan bare returnere hvis
    // vi ser et. løkken nedenfor går gjennom alle
    for (int i = 0; i < n-1; i++)
        for (int j=i+1; j < n; j++)
            if (check_overlap(arr[i], arr[j])) return 1;

    // kommer vi hit betyr det at alle check_overlap testene var 0
    return 0;
}
```